Multimodal AI Teaching Copilot for Real-Time Visual Learning Generation

Prachi Potdar, Paritosh Raut, Deepali Magdum, Swati Kale, and Sunita Patnaik
Department of Electronics and Telecommunication Engineering
JSPM's Rajarshi Shahu College of Engineering, Pune, Maharashtra, India – 411033
prachipotdar50@gmail.com, paritosh21raut@gmail.com, dipalimagdum@gmail.com, swatikale@gmail.com, sunitapatnaik@gmail.com

***Abstract*—Lecture slides are prepared before class and cannot follow what a teacher actually explains, while live captions follow the speech but show only text. We describe the design and implementation of a teaching copilot that listens to a live lecture and builds a structured slide deck on the classroom projector while the teacher speaks. Speech is transcribed locally with a Whisper-based recognizer. A sentence-embedding concept tracker and a rule-based gate decide when a bounded unit of transcript is sent to a large language model (LLM), which returns a schema-validated description of topic, subtopic, discourse relation and content. A deterministic planner, not the LLM, decides whether to update the live slide, continue on a new slide or open a new topic, using confirmation and dwell rules that keep the display stable. A composer maps the content to designed layouts with typeset formulas and openly licensed images ranked with CLIP; the teacher can pause, edit, navigate and replace images from a control view, and can turn a finished lecture into summary slides, notes, an assignment and a PPTX deck. In six live lectures (114 min) covering physics, chemistry, biology, mathematics and engineering topics, run on a laptop with a 4 GB GPU and free services, the system built 163 slides with median speech-to-text and LLM latencies of 0.66 s and 1.35 s; 344 of 389 units were interpreted by the LLM and the rest, during quota exhaustion, by a deterministic fallback. Concept simulation and narrative (video) visualization are planned as further modes.**

***Index Terms*—Educational technology, speech recognition, large language models, automatic slide generation, real-time systems, multimodal learning, human-in-the-loop**

I. Introduction

Slides prepared before a lecture fix the order and depth of the material in advance, so examples, clarifications and answers given during the class do not appear on the screen. Research on multimedia learning shows that students learn better from words combined with well-organized pictures than from words alone [1], [2], provided that the visuals reduce rather than add extraneous load [3], [4]. A display that mirrors the spoken explanation in structured visual form is therefore attractive, but producing it by hand during a lecture is not practical.

Speech recognition [5] and large language models (LLMs) [6], [7] now make it possible to follow a lecture as it happens. Used naively, however, they give an unstable display: an LLM that re-generates slides from a growing transcript reorders, renames and rewrites content every few seconds, adds facts that were never taught [8], and exhausts free-tier quotas within minutes. Live captioning avoids these problems but shows only text, without the structure of the lesson [9].

This work asks how a live, continuity-aware slide display can be generated from a teacher's speech on modest hardware with free services, while the teacher stays in control. The contributions are: (1) an implemented end-to-end copilot that turns live lecture speech into projected slides during the lecture; (2) an architecture in which the LLM is a gated, bounded and validated reasoning service inside a deterministic event-driven core, so that slide lifecycle decisions are testable rules; (3) a planner and composer that separate *what was said* from *how it is shown*; (4) teacher tools for live correction and for post-lecture materials; and (5) an evaluation on six live lectures (114 min) across physics, chemistry, biology, mathematics and engineering topics, with a clear account of what is not yet implemented.

II. Related Work

A. Multimedia Learning and Slide Design

Paivio's dual coding theory holds that verbal and pictorial information are processed in two connected systems [2]. Building on it, Mayer showed in a series of experiments that learners understand an explanation better from words and pictures together and derived design principles such as coherence and signalling [1]. Sweller's cognitive load theory adds that working memory is limited, so extraneous material hinders learning [3], and Mayer and Moreno described nine ways to reduce this load in multimedia lessons [4]. In a classroom comparison, Bartsch and Cobern found that students liked PowerPoint lectures but that pictures unrelated to the content lowered their performance [10]. These findings shape our design: short typed layouts, and images only where they show something concrete.

B. Lecture Speech Recognition and Captioning

Radford *et al.* trained Whisper on 680,000 hours of weakly supervised audio and obtained robust recognition without domain-specific fine-tuning [5]; the faster-whisper project re-implements it on CTranslate2 with 8-bit weights, which makes it fast enough for a consumer GPU [11], and Silero VAD provides a compact pre-trained voice activity detector [12]. Glass *et al.* used recognition to transcribe and index recorded lectures for search and browsing [13]. Kushalnagar *et al.* evaluated real-time classroom captions with deaf and hard-of-hearing students and reported that verbatim text at speaking rate is difficult to follow [9]. These systems produce text; they do not restructure what is said into a visual lesson.

TABLE I
COMPARISON WITH EXISTING APPROACHES

Approach	Input	When produced	Lecture continuity	Teacher control	Visual structure
Prepared slides (conventional)	Teacher authoring	Before class	Fixed in advance	Full, but offline	As designed
Live captioning [5], [9]	Speech	Live	None (text stream)	Little	Plain text
Document-to-slides [18]–[20]	Written document	Offline	Document sections	Edit afterwards	Text + figures
Prompt-based LLM slide tools	Typed prompt	Offline, on request	None across a lecture	Edit afterwards	Template layouts
Proposed copilot	Live speech	During the lecture	Tracked topic / subtopic, planner rules	Live: pause, edit, navigate, images	Typed layouts, formulas, images

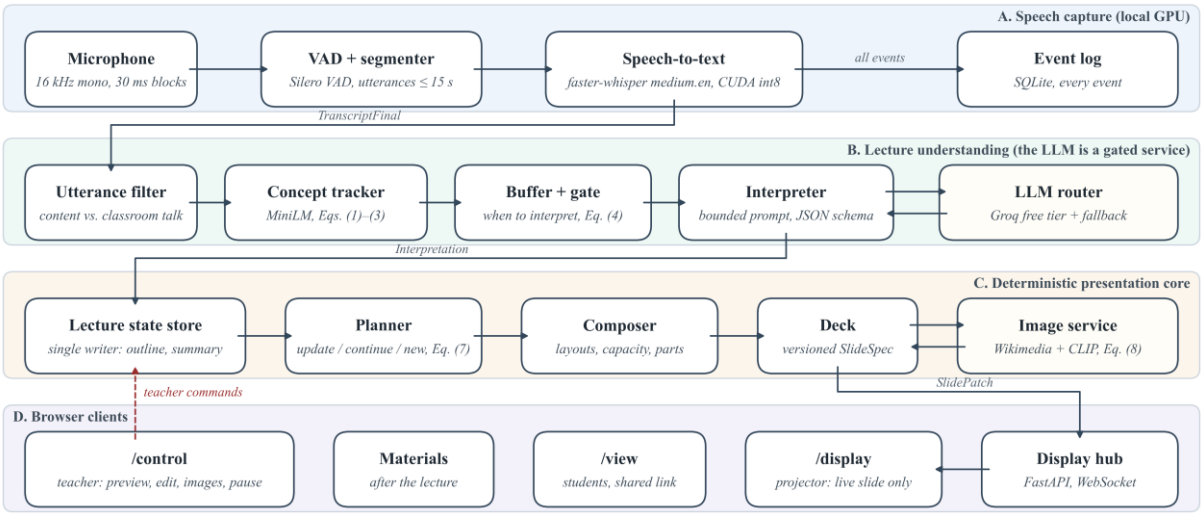


Fig. 1. Implemented architecture. Lanes A–C run in one Python process; the LLM is called only by the interpreter through the router. Browser clients receive versioned slide patches and send teacher commands back; the user interface never changes the core state directly.

C. Topic Segmentation and Semantic Similarity

Hearst's TextTiling places subtopic boundaries where the lexical similarity of adjacent text blocks drops [14], and Malioutov and Barzilay segmented spoken lectures by a minimum-cut partition of a sentence similarity graph [15]. Reimers and Gurevych introduced Sentence-BERT, whose sentence embeddings can be compared with cosine similarity [16], and Wang *et al.* distilled such models into the compact MiniLM [17]. These methods work on a complete transcript; the copilot applies the same similarity idea online, one utterance at a time, as one signal among several.

D. Automatic Slide Generation

Hu and Wan proposed PPSGen, which learns sentence importance and assembles slides for academic papers by integer linear programming [18]. Sun *et al.* presented D2S, which uses slide titles as queries to retrieve and summarize the matching document text [19], and Fu *et al.* introduced DOC2PPT, which generates slides with text and figures from scientific documents [20]. All three start from a finished written document and run offline, whereas a lecture arrives incrementally as noisy speech and its slides must stay stable while students watch.

E. Large Language Models in Education

Kasneci *et al.* discussed the opportunities of LLMs for explanation and material generation and the need for teacher oversight [6]. Ji *et al.* surveyed hallucination, where generated text is not supported by the source [8], and Willard and Louf showed that generation can be constrained efficiently to a JSON schema [21], which fixes the format but not the facts. The copilot therefore asks the LLM only for a bounded, structured interpretation of what was just said, checks it against the transcript and leaves layout and slide lifecycle to deterministic code. Table I contrasts the approach with existing workflows.

III. METHODOLOGY

The copilot treats a lecture as a stream that passes through six stages (Fig. 1): (1) speech is captured and transcribed locally; (2) classroom talk is filtered out and the current concept is tracked; (3) a gate groups finished sentences into units worth interpreting; (4) an LLM interprets each unit within a bounded context; (5) a deterministic planner and composer turn the interpretation into slide operations and layouts; and (6) the slides reach the projector, the teacher and the students, and on request the stored lecture becomes study material. Four rules guide the design: the LLM never controls the flow, it never sees the whole transcript, a new concept does not by itself mean a new slide, and user interfaces send commands instead of changing the state.

All components run in one Python *asyncio* process and communicate only through typed events on an in-process bus. A single store owns the lecture state, and every event is appended to an SQLite log, so each lecture can be replayed for

debugging and for materials. Models run in worker threads with bounded queues, so audio capture never blocks. Rendering is progressive (a quick deterministic update, then the refined interpretation in place, then images), and teacher commands override automatic decisions. Only free or local components are used: faster-whisper on the GPU, MiniLM and CLIP as ONNX models on the CPU, FastAPI with WebSocket, and browser clients with Preact, KaTeX and SVG. The LLM (gpt-oss-120b [7] with a Qwen backup) runs on the Groq free tier, because a 7B-class model does not fit beside the speech model in 4 GB of GPU memory.

IV. Implementation Details

A. Speech Capture and Concept Tracking

Audio is captured at 16 kHz in 30 ms blocks. Silero VAD segments speech (600 ms hangover, utterances capped at 15 s), faster-whisper transcribes each utterance with beam size 1 and a short subject-keyword prompt, and likely hallucinations are discarded. A deterministic filter then separates content from classroom management ("take out your books"), meta talk and fillers. Each content utterance is embedded with MiniLM [17] (about 6 ms on CPU). With the L2-normalized embedding e_t and concept centroid c_{t-1}, the shift score (a cosine distance [16]) and the centroid update are

$$s_t = 1 - \mathbf{e}_t^{\top}\mathbf{c}_{t-1} \quad (1)$$

$$\mathbf{c}_t = \mathrm{norm}((1-\alpha_t)\mathbf{c}_{t-1} + \alpha_t \mathbf{e}_t) \quad (2)$$

where norm(x) = $x/\|x\|$ and $\alpha = 0.35$, raised to 0.6 at a boundary so that the centroid re-seeds on the new concept. Similar to the similarity-drop criterion of TextTiling [14], a boundary is signalled when

$$b_t = \kappa_t \vee (s_t \geq \tau), \tau = 0.75 \quad (3)$$

where κ_t marks a strong cue phrase ("now let us", "next", "is defined as"). Single-sentence shifts were noisy (0.3–0.6 within one facet), so τ is high, cue words carry most boundary detection and the LLM decides the final relation.

B. Gate and Token Budget

Content accumulates in a buffer until the gate decides that a meaningful unit is ready. With p_t the VAD silence after a finished sentence, w_t the buffered words and a_t the age of the oldest pending line (in seconds), the gate fires when

$$G_t = (p_t \geq 1.2 \wedge w_t \geq 6) \vee w_t \geq 40 \vee b_t \vee a_t \geq 12 \quad (4)$$

subject to an 8 s floor between calls. New lines queue while a call is in flight, units end only after finished sentences, and fragments under six words join the next unit. The floor follows from the free-tier budget: with daily quota Q per key and model, system prompt P_s, dynamic context P_d and output O,

$$N_{\mathrm{day}} \approx Q/(P_s + P_d + O) \approx 200000/2300 \approx 85 \quad (5)$$

or about 15–20 lecture minutes per key; calling the LLM on every utterance would exhaust it several times faster.

C. Bounded and Truthful Interpretation

The prompt holds a header (subject, level, an outline of at most 150 tokens), the current slide with numbered items, a rolling summary of at most 120 tokens and the new lines, within a 3,000-token budget. The LLM returns JSON with topic, subtopic, a relation (same concept, elaboration, sub-concept, sibling, new topic, digression), typed content items (definition, facts, classification, steps, comparison, timeline, formula, cause-effect, example), an image hint, revisions and *concerns* about possibly wrong statements. Pydantic validates the answer; after one failed repair, a logged deterministic fallback shows the spoken sentences as points. A coverage guard compares every complete content sentence u (at least six words) with the model output O; with $W(\cdot)$ the set of word stems, a recall-style overlap as in ROUGE-1 [22] gives

$$|W(u) \cap W(O)| \leq \tfrac{1}{3}|W(u)| \Rightarrow u \text{ is shown as said} \quad (6)$$

so a fact the model left out still reaches the slide. A grounding guard flags formula tokens that the model changed from what was heard. A correction is applied only above a confidence threshold (0.75 factual, 0.4 mis-hearing); otherwise the slide shows what the teacher said, and for factual concerns the teacher gets a card with both versions. The router adds token buckets, a daily usage ledger, several free keys used in turn and a 15 s deadline per unit.

D. Slide Planner

The planner turns each interpretation into one operation: *update* the working slide, *continue* on a new slide of the same topic (new facet or further part), open a *new* topic slide, *retitle*, or do nothing (Fig. 2). An elaboration or sub-concept usually updates the live slide while it has room. A topic change must be confirmed, because one unit can be misread:

$$C_n = B_n \vee \left(\bar{s}_n^{\max} \geq \tau\right) \vee \left(T_n \equiv \tilde{T}_{n-1}\right) \quad (7)$$

where B_n is true if a boundary (3) occurred in unit n, $\bar{s}_n^{\max}$ is the largest shift score in the unit smoothed over two utterances, T_n the topic of unit n and $\tilde{T}_{n-1}$ an unconfirmed candidate from the previous unit. Unconfirmed content is shown tentatively and moved once confirmed. A minimum dwell of 15 s (6 s for the next part of a full slide) prevents rapid changes, near-duplicates (text similarity ≥ 0.85) are merged, and teacher edits are never overwritten.

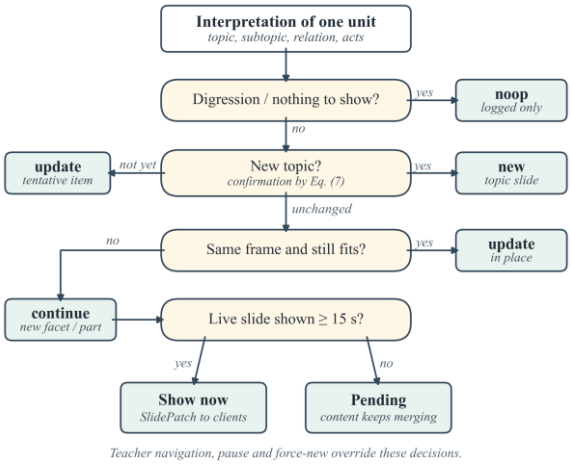


Fig. 2. Planner decision for one interpreted unit. The LLM supplies the relation and content; the decision itself is rule-based and unit-tested.

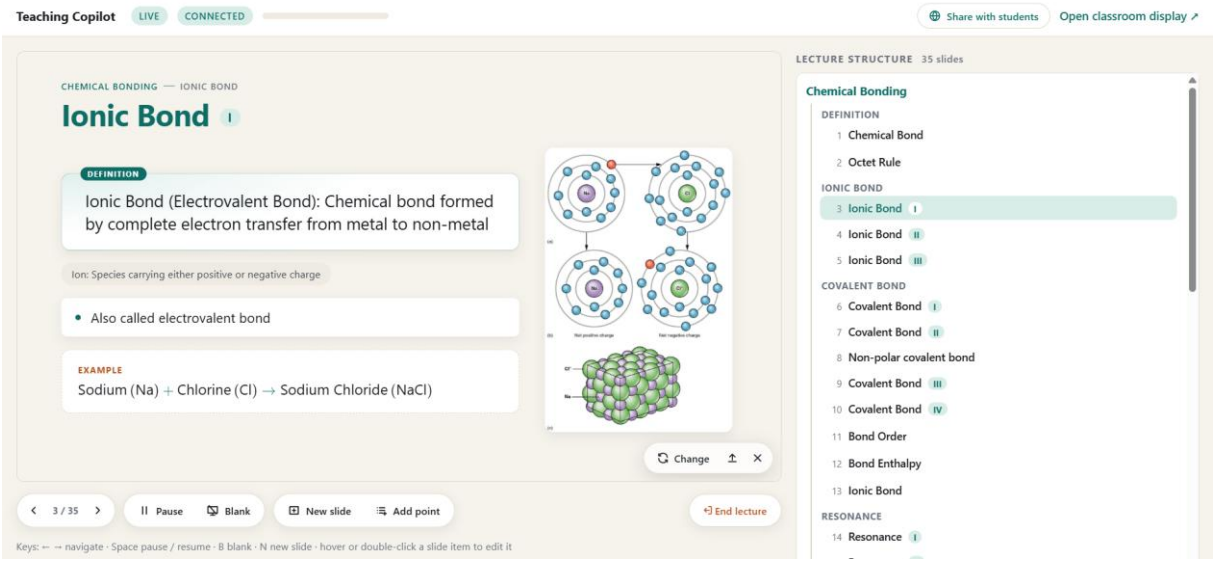


Fig. 3. Teacher control view (/control) during a live chemistry lecture. Left: editable preview of the projected slide with a definition card, a term chip, a key point, a worked example and an automatically retrieved labelled diagram, with image tools and lecture controls below. Right: the lecture structure tree built by the system (topic → facet → numbered slides and parts).

E. Composer and Image Retrieval

The composer maps items to layout blocks: definition cards, key points, fact tiles, group cards, process flows (up to ten steps in two rows), comparison tables, timelines, cause-effect chains, classification trees and formulas. A pixel-level height model of the 16:9 stage decides what fits; overflow moves to a further part of the slide (I, II, …) instead of shrinking the text. Spoken mathematics is converted to LaTeX deterministically ("v equals under root 2RT upon m" becomes a square-root fraction) and typeset with KaTeX with a variable legend. Concrete objects and structures get an image: for a query q suggested by the LLM, the service searches Wikipedia and Wikimedia Commons, filters by licence, size and English labels, and scores up to eight previews with CLIP [23] on the CPU. Image I is accepted when

$$\sigma(q,I) \geq 0.22 \wedge \sigma(q,I) - \max_{d \in D} \sigma(d,I) \geq 0.01 \quad (8)$$

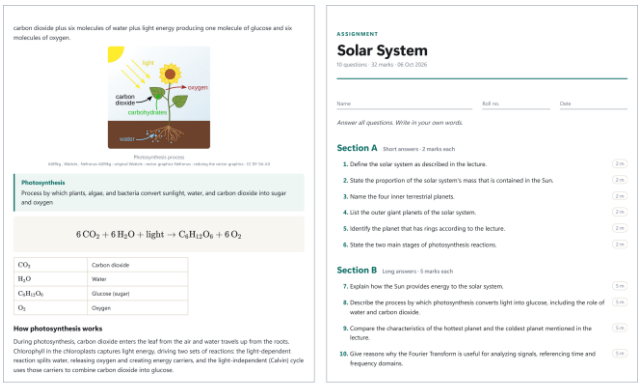
where σ is the CLIP cosine similarity and D a set of generic distractors ("a logo", "a map"). Labelled diagrams rank first for structures, and an image is placed only if the MiniLM similarity of query and slide text is at least 0.25.

F. Live Display and Teacher Control

A FastAPI server pushes versioned slide patches over WebSocket to /display (projector, live slide only), /control (teacher) and /view (students on a shared link); a display reconnects and requests the current state after a network drop. The control view (Fig. 3) offers an editable preview, navigation, pause (speech during a pause is not interpreted), blank screen, new slide, in-place editing, image search, upload and removal, mistake cards, the lecture structure tree and the latest transcript line. Slides can switch between light and dark themes mid-lecture, and the control view has its own dark mode.

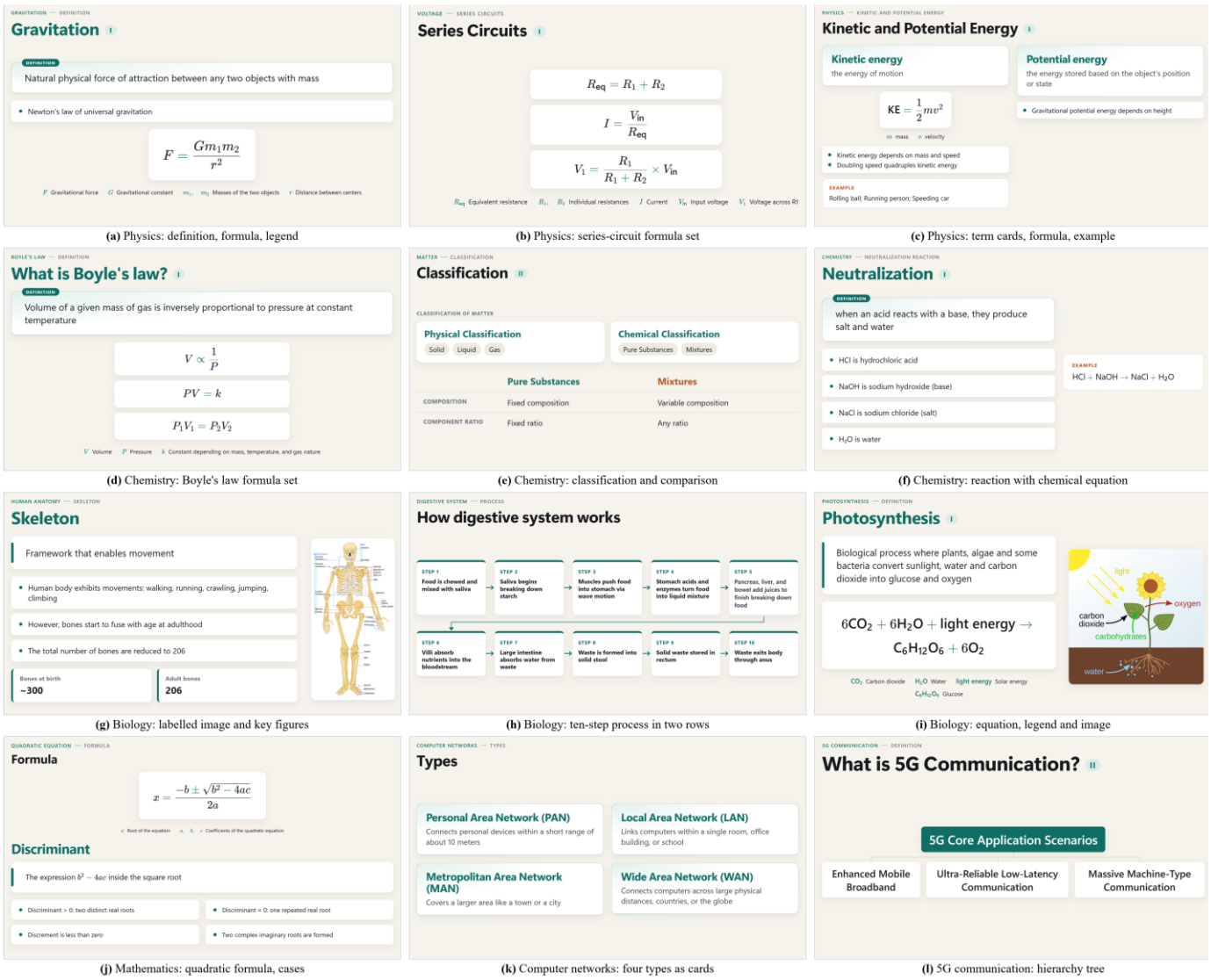
G. Lecture Materials and Archive

Only when the teacher asks, the stored lecture is turned into teaching material: a summary slide for the current topic or the whole lecture, a key-concepts slide, study notes as a PDF that expand the slides and transcript into written explanations, an assignment PDF with short and long theory questions (ten by default) and a PPTX copy of the deck in the light or dark theme. Materials can draw on the current lecture, selected past lectures or a whole chapter. Lectures are archived in chapters named by the teacher, can be searched by title and slide content and reopened in /control, and students download released materials through the shared link. With *My notes*, the teacher keeps a reference file (PDF, Word, PowerPoint, text or a web page) beside the preview; a MiniLM page matcher follows the lecture to the relevant page without LLM tokens, and a page can be projected on demand. Fig. 4 shows notes and an assignment generated this way.



(a) Study notes (PDF, page 3) (b) Assignment (PDF)

Fig. 4. Materials generated on request from a recorded lecture with the real LLM: (a) a study-notes page with a retrieved diagram, definition, typeset equation and legend; (b) an assignment with short and long theory questions grounded in what was taught.



(a) Physics: definition, formula, legend **(b)** Physics: series-circuit formula set **(c)** Physics: term cards, formula, example

(d) Chemistry: Boyle's law formula set **(e)** Chemistry: classification and comparison **(f)** Chemistry: reaction with chemical equation

(g) Biology: labelled image and key figures **(h)** Biology: ten-step process in two rows **(i)** Biology: equation, legend and image

(j) Mathematics: quadratic formula, cases **(k)** Computer networks: four types as cards **(l)** 5G communication: hierarchy tree

Fig. 5. Projected slides from the six live lectures, grouped by subject (light theme). (g), (i), (e) and (j) are screenshots of the running application; the others are the recorded LLM interpretations of the same lectures re-rendered by the current display engine without new LLM calls. Slide text is as spoken, including the speaker's own slips such as "Discrement" in (j).

V. Experimental Evaluation

A. Setup

All runs used a laptop with an NVIDIA RTX 3050 Laptop GPU (4 GB) and 16 GB RAM, and the free tier of Groq. Six lectures were spoken live by the authors into the laptop's microphone with /control and /display open, during development of the system. They covered physics, chemistry, biology, mathematics, computer networks and communication engineering, in single-subject and mixed sessions. When the main model's daily quota was spent, the router served the Qwen backup. All numbers come from the application's event logs; no user study was conducted.

B. Results Across Six Live Lectures

Table II summarizes the sessions. In 114 min of speech, 765 transcribed utterances formed 389 interpretation units, and the system built 163 slides. The LLM interpreted 344 units; all 45 fallback units occurred while every key's daily quota was spent, and they still showed the spoken sentences. In the fifth session the teacher paused the copilot for long stretches, so few units were interpreted.

Fig. 5 shows twelve projected slides from these lectures, grouped by subject. In physics the copilot typeset spoken formulas with variable legends, from a single law to a set of circuit equations; in chemistry it produced formula sets, a classification with a comparison table and a reaction with its chemical equation; in biology it placed labelled diagrams beside definitions and key figures and laid out a ten-step process in two rows; and for mathematics and engineering it built a formula with its cases, type cards and a hierarchy tree. The layout follows the kind of content, not the subject.

TABLE II
SIX LIVE LECTURES (SESSION LOGS)

Lecture topics	Min	Utt.	Units (fb)	STT	LLM	Slides	Images
Phys., maths, chem.	26.1	121	64 (6)	0.73	1.29	32	–
Biology (anatomy)	12.5	67	45 (0)	0.74	1.34	11	7 / 11
Bio., phys., engg.	20.3	159	89 (2)	0.62	1.39	43	6 / 13
CS, maths, bio.	24.2	157	87 (30)	0.61	1.20	32	10 / 12
Chem., bio., phys.	13.3	119	16 (0)	1.00	1.56	10	2 / 2
Chemistry	17.6	142	88 (7)	0.66	1.30	35	8 / 25
All six	**114.1**	**765**	**389 (45)**	**0.66**	**1.35**	**163**	**33 / 63**

Utt.: transcribed utterances; fb: fallback units; STT and LLM: median latency in seconds; Images: searches with an accepted image / searches (none in the first session).

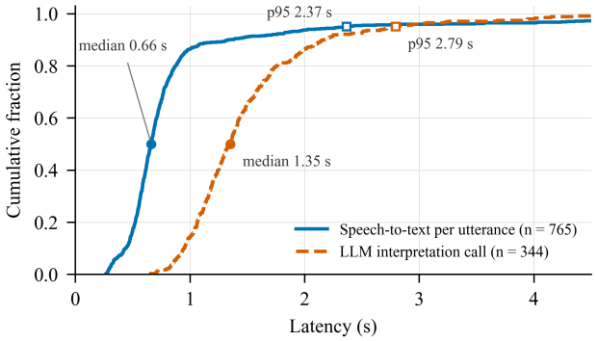


Fig. 6. Cumulative distribution of speech-to-text latency per utterance and LLM call latency per interpretation, pooled over the six live lectures. Filled markers: medians; open: 95th percentiles.

Fig. 6 shows the pooled latencies. Speech-to-text has a median of 0.66 s and stays under one second for 86 % of utterances; the tail comes mainly from the fifth session, where recognition slowed to 4–10 s for some segments, while in the other five sessions 95 % of utterances took at most 1.4 s. LLM calls have a median of 1.35 s and a 95th percentile of 2.79 s, so the model is not the main source of delay.

C. Detailed Session: Chemistry Lecture

The chemistry lecture (last row of Table II) moved from chemical bonding to the gas laws and the kinetic theory of gases. The planner made 55 in-place updates, 12 continuations and 7 new-topic decisions, and 14 units changed nothing (digressions, announcements, content already shown). Three topic proposals were held as tentative until confirmed by (7), and the final deck had 35 slides (Fig. 3, Fig. 7). Twenty-three mis-hearings were corrected silently, e.g. "collide with the world of container" shown as "walls of container". A line waited a median of 11 s before being sent, because every concept boundary produced a short unit that queued behind the 8 s floor; with units that end only after finished sentences, a replay of this lecture reduced the median wait from 9.5 s to 5.3 s and requests from 107 to 83 (verified in simulation, not yet live). Most failed image searches were queries with no matching Commons file name ("pi bond orbital overlap"); a core-phrase search added later found images for 9 of the 16 failed queries.

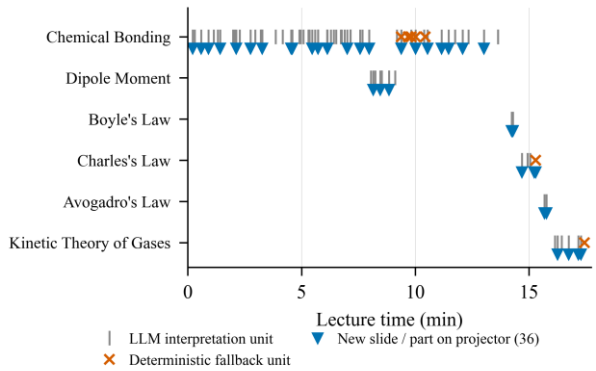


Fig. 7. Topics detected over the chemistry lecture: LLM units (ticks), fallback units (crosses) and new slides or parts (triangles).

D. Component Measurements

Whisper medium.en transcribes a 14 s clip in 0.75 s using about 1.1 GB of GPU memory; CLIP scores eight images in about 0.69 s on the CPU. On a 30-concept labelled set, the first returned image was relevant for all 28 concepts that returned one. For lecture materials, a run on a recorded lecture (topic and lecture summary, key concepts, notes and ten questions) used 7,021 LLM tokens, and a PPTX export took 2.9 s. All 513 unit and integration, 28 browser and 8 GPU tests pass.

VI. DISCUSSION

Unlike the approaches in Table I, the copilot combines live input, a stable deck that follows the teacher's sequence and a human in the loop. Deterministic lifecycle decisions made the behaviour explainable: every slide change in the log carries a reason, and issues found in live tests could be reproduced by replaying the recorded session at zero LLM cost. The bounded prompt and gate make a free-tier LLM usable, but only for limited lecture time per day, and the delay seen in class depends more on unit formation and rate limits than on the models. The lectures were given by the authors in English during development, without a classroom study, so no claim is made about learning outcomes; subtopic naming still varies between runs, image coverage depends on Wikimedia, and crash recovery and a 60-minute soak test are designed but not implemented.

VII. PLANNED EXTENSIONS

Because understanding is separate from rendering, further modes reuse the lecture state and add only their own planner, composer and display components. They are planned and not yet implemented. **Live concept visualization.** For mechanisms with motion or forces, a slide block will host an interactive simulation in the spirit of PhET [24]; developers write parametric templates (incline, pulley, circuit, wave, orbit, cell cycle), and the LLM only selects a template and fills validated parameters, so no unimplemented physics can appear. **Narrative visualization.** For historical or story-like content, the copilot will show a short visual narrative synchronized with the narration. Open text-to-video models [25], [26] need 8–24 GB of GPU memory and minutes per

clip, so the first version is a storyboard of three to six scenes from retrieved or teacher images with camera motion and captions; generated video is a later upgrade behind the same scene specification. Quizzes and tests through the shared link, Hindi and Marathi input, and a classroom study are also planned.

VIII. Conclusion

This paper presented an implemented teaching copilot that generates a structured, continuity-aware slide deck on the projector while the teacher speaks, treating the LLM as a bounded, validated interpreter inside a deterministic, event-driven core, and that turns the lecture into summaries, notes, assignments and slides on request. On a laptop with a 4 GB GPU and free services, it followed six live lectures across five subject areas, built 163 slides with median speech-to-text and LLM latencies of 0.66 s and 1.35 s, and kept the teacher in control. Future work will measure its effect in real classes and add concept simulation and narrative visualization on the same core.

Acknowledgment

We are grateful to the Department of Electronics and Telecommunication Engineering, JSPM's Rajarshi Shahu College of Engineering, Pune, for the guidance and facilities provided for this work.

References

[1] R. E. Mayer, *Multimedia Learning*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[2] A. Paivio, *Mental Representations: A Dual Coding Approach*. New York, NY, USA: Oxford Univ. Press, 1986.

[3] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Sci.*, vol. 12, no. 2, pp. 257–285, 1988.

[4] R. E. Mayer and R. Moreno, "Nine ways to reduce cognitive load in multimedia learning," *Educ. Psychologist*, vol. 38, no. 1, pp. 43–52, 2003.

[5] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 28492–28518.

[6] E. Kasneci *et al.*, "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individual Differences*, vol. 103, Art. no. 102274, 2023.

[7] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," arXiv:2508.10925, 2025.

[8] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 248, 2023.

[9] R. S. Kushalnagar, W. S. Lasecki, and J. P. Bigham, "Accessibility evaluation of classroom captions," *ACM Trans. Accessible Comput.*, vol. 5, no. 3, Art. no. 7, 2014.

[10] R. A. Bartsch and K. M. Cobern, "Effectiveness of PowerPoint presentations in lectures," *Comput. Educ.*, vol. 41, no. 1, pp. 77–86, 2003.

[11] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," GitHub repository. [Online]. Available: https://github.com/SYSTRAN/faster-whisper

[12] Silero Team, "Silero VAD: Pre-trained enterprise-grade voice activity detector," GitHub repository, 2021. [Online]. Available: https://github.com/snakers4/silero-vad

[13] J. Glass, T. J. Hazen, S. Cyphers, I. Malioutov, D. Huynh, and R. Barzilay, "Recent progress in the MIT spoken lecture processing project," in *Proc. Interspeech*, 2007, pp. 2553–2556.

[14] M. A. Hearst, "TextTiling: Segmenting text into multi-paragraph subtopic passages," *Comput. Linguistics*, vol. 23, no. 1, pp. 33–64, 1997.

[15] I. Malioutov and R. Barzilay, "Minimum cut model for spoken lecture segmentation," in *Proc. 21st Int. Conf. Comput. Linguistics and 44th Annu. Meeting Assoc. Comput. Linguistics (COLING-ACL)*, 2006, pp. 25–32.

[16] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[17] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 5776–5788.

[18] Y. Hu and X. Wan, "PPSGen: Learning-based presentation slides generation for academic papers," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 4, pp. 1085–1097, Apr. 2015.

[19] E. Sun, Y. Hou, D. Wang, Y. Zhang, and N. X. R. Wang, "D2S: Document-to-slide generation via query-based text summarization," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics (NAACL-HLT)*, 2021, pp. 1405–1418.

[20] T.-J. Fu, W. Y. Wang, D. McDuff, and Y. Song, "DOC2PPT: Automatic presentation slides generation from scientific documents," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 1, 2022, pp. 634–642.

[21] B. T. Willard and R. Louf, "Efficient guided generation for large language models," arXiv:2307.09702, 2023.

[22] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Proc. ACL Workshop Text Summarization Branches Out*, 2004, pp. 74–81.

[23] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 8748–8763.

[24] C. E. Wieman, W. K. Adams, and K. K. Perkins, "PhET: Simulations that enhance learning," *Science*, vol. 322, no. 5902, pp. 682–683, 2008.

[25] J. Ho, T. Salimans, A. Gritsenko, W. Chan, M. Norouzi, and D. J. Fleet, "Video diffusion models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 8633–8646.

[26] A. Blattmann *et al.*, "Stable video diffusion: Scaling latent video diffusion models to large datasets," arXiv:2311.15127, 2023.